

Deepfake Detection

Final Project Report

Ajitesh Kumar Singh (IMT2022559)
Vaibhav Bajoroya (IMT2022574)

Abstract

Deepfakes have emerged as a serious threat to digital trust by enabling highly realistic manipulation of visual and audio media. Existing detection methods are often offline, unimodal, or unsuitable for deployment in practical user environments. To address this limitation, we propose a real-time multimodal deepfake detection framework designed for seamless operation on mobile devices. The proposed system performs operating-system-level media acquisition, enabling application-independent capture of both on-screen visual content and internal audio streams without interrupting user activity. For visual forgery analysis, a hybrid **GenConViT** architecture integrating autoencoders, variational autoencoders, ConvNeXt, and Swin Transformer modules is employed, achieving a classification accuracy of **92.1%**. Audio spoof detection is carried out using a fine-tuned **Wav2Vec2** model, which attains **99.7%** accuracy on synthetic speech datasets. Outputs from both modalities are combined through a confidence-aware late fusion strategy, further strengthened by a reinforcement learning mechanism that adaptively updates modality importance using user feedback. Experimental evaluation demonstrates superior generalization and robustness over conventional CNN- and Vision Transformer-based baselines in realistic usage scenarios. The proposed framework provides a scalable, adaptive, and deployable solution for deepfake detection directly at the point of media consumption.

1 Introduction

Deepfakes have become increasingly realistic due to rapid advancements in generative models, making it difficult to distinguish between authentic and manipulated media. This creates serious concerns around misinformation, identity misuse, and trust in digital content.

The goal of this project was to build a **practical deepfake detection system** that works in real-world scenarios, particularly on mobile devices where most media is consumed. Unlike traditional approaches that operate on pre-collected datasets, this system is designed to detect deepfakes **in real time while the user is consuming content**.

To achieve this, we developed a **multimodal pipeline** that captures both:

- Visual data (screen frames)

- Audio data (internal device audio or microphone fallback)

The captured data is processed using deep learning models and the results are displayed to the user via a floating overlay without interrupting normal device usage.

During development, multiple model architectures and approaches were explored. Several initial approaches did not perform well due to issues such as poor generalization, sensitivity to compression artifacts, and inability to handle real-world data. These failures guided the design of more robust models, including hybrid architectures that combine convolutional networks, transformers, and generative models.

This report documents:

- The overall system design
- The frontend Android pipeline
- The different model architectures that were tried
- Analysis of why certain approaches failed
- The final working models and why they performed better
- Additional explorations, including multimodal LLM-based approaches

The focus of this report is not only on the final results, but also on the **engineering decisions and iterative improvements** that led to the current system.

2 Deepfake Detection

Problem Definition

The deepfake detection objective can be formulated as a supervised binary classification problem, where the system determines whether a given input media sample belongs to the authentic or manipulated class. Mathematically, this decision process is represented as:

$$f(x) \rightarrow \{\text{Real}, \text{Fake}\}$$

where x denotes the input image (or extracted media frame), $f(\cdot)$ represents the learned classification function, and the output label indicates whether the content is genuine (*Real*) or forged (*Fake*).

3 Dataset Description

The primary dataset used for visual models is the OpenDeepfake-Preview dataset, curated for training and evaluating deepfake detection models. It consists of approximately 20,000 labeled image samples with binary annotations (0: Fake, 1: Real).

Key Factors in Deepfake Detection

Reliable deepfake detection depends on the identification of subtle visual and temporal anomalies introduced during synthetic media generation. These irregularities often remain imperceptible to human observers but can be effectively captured through computational feature analysis. The principal indicators exploited in modern deepfake detection systems are summarized below:

- **Facial Texture Inconsistencies:** Presence of unnatural skin smoothing, irregular pore structures, over-blending, or synthetic texture distortions.
- **Spatial Artifacts:** Localized abnormalities around facial contours, eye boundaries, lip regions, and hair edges caused by imperfect face synthesis or swapping.
- **Lighting and Shadow Errors:** Mismatch in illumination intensity, shadow orientation, or reflectance properties across different facial areas.
- **Frequency-Domain Disturbances:** Unusual high-frequency spectral signatures and repetitive generative noise patterns introduced by deep synthesis networks.
- **Biological Signal Deviations:** Missing or inconsistent natural cues such as blinking behavior, pulse variations, eye movement, and micro-expressions in video content.
- **Temporal Inconsistencies:** Frame-wise irregularities in facial alignment, motion continuity, or expression transitions across consecutive video frames.

4 System Setup and Backend Environment

The backend system is deployed using a **Conda environment** to manage dependencies and ensure reproducibility. The environment relies on Python 3.10 and includes critical machine learning and server libraries.

4.1 Environment Configuration

The setup process involves creating a Conda environment from the provided `environment.yml` file. Key dependencies include:

- **Deep Learning:** `torch` (2.10.0), `transformers` (5.3.0), `tensorflow` (2.21.0)
- **Web Server:** `fastapi` (0.135.1), `uvicorn` (0.42.0), `websockets` (16.0)
- **Media Processing:** `opencv-python`, `librosa`, `moviepy`, `pillow`

The environment is initialized and the server is started using Uvicorn:

```
conda env create -f environment.yml
conda activate deepfake
uvicorn server:app --host 0.0.0.0 --port 7860
```

5 System Overview

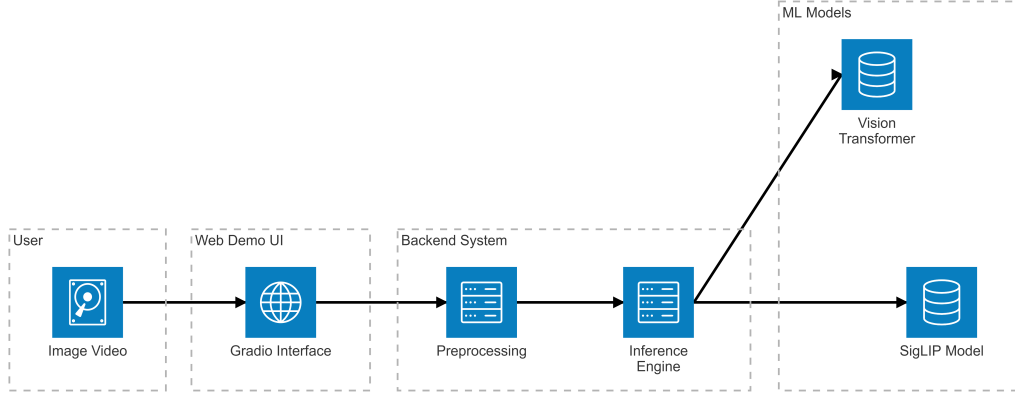


Figure 1: Comprehensive Project Design Overview

The developed system is a real-time multimodal deepfake detection pipeline designed to operate on mobile devices without requiring integration with individual applications. The system captures media content at the operating system level, processes it using backend models, and provides continuous feedback to the user.

5.1 Operational Modes: Static vs. Real-Time

During the early phases of the project, two distinct architectural workflows were prototyped: a static image analyzer and a real-time stream processor.

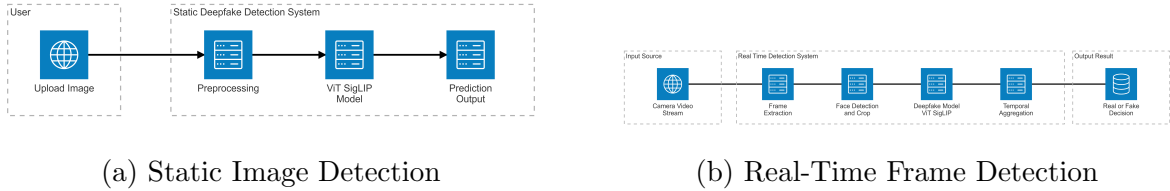


Figure 2: Comparison of Static vs. Real-Time Architectures

5.2 High-Level Pipeline

The overall workflow of the system can be summarized as follows:

1. Screen frames and audio are captured from the device in real time.
2. The captured data is preprocessed into standardized formats.
3. The data is transmitted to a backend server for inference.
4. Deep learning models analyze the visual and audio inputs.
5. The prediction (real or fake) is returned along with a confidence score.

6. The result is displayed to the user through a floating overlay.

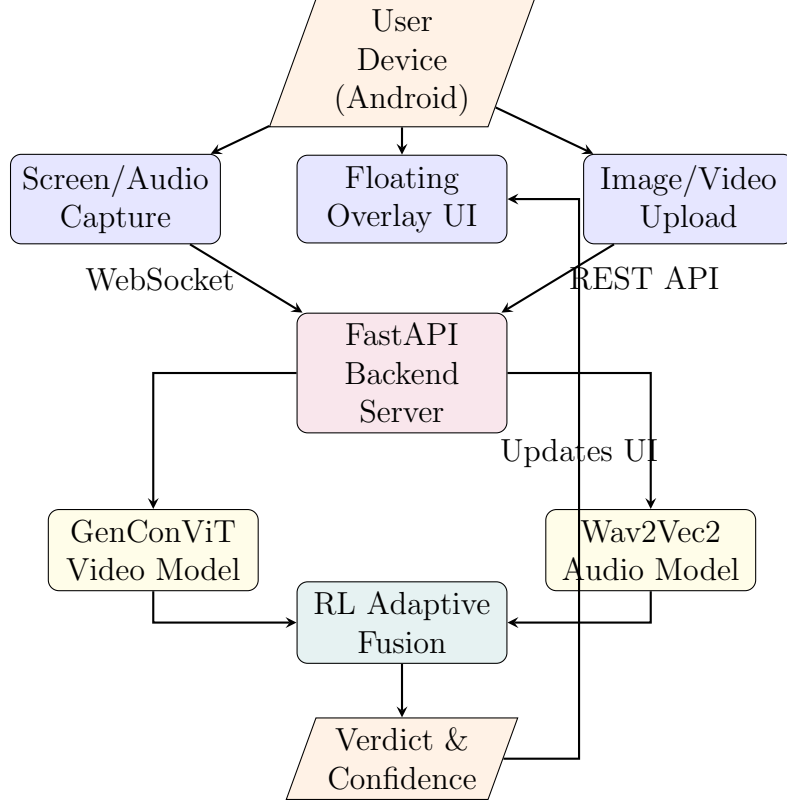


Figure 3: High-Level System Design Flowchart

This pipeline enables continuous monitoring of media content across arbitrary applications such as video players, messaging apps, and web browsers.

5.3 Media File Upload Analysis

In addition to real-time streaming, the system natively supports the explicit upload of images and videos for deepfake analysis. This feature allows users to verify specific files (e.g., received via messaging apps) without requiring continuous monitoring.

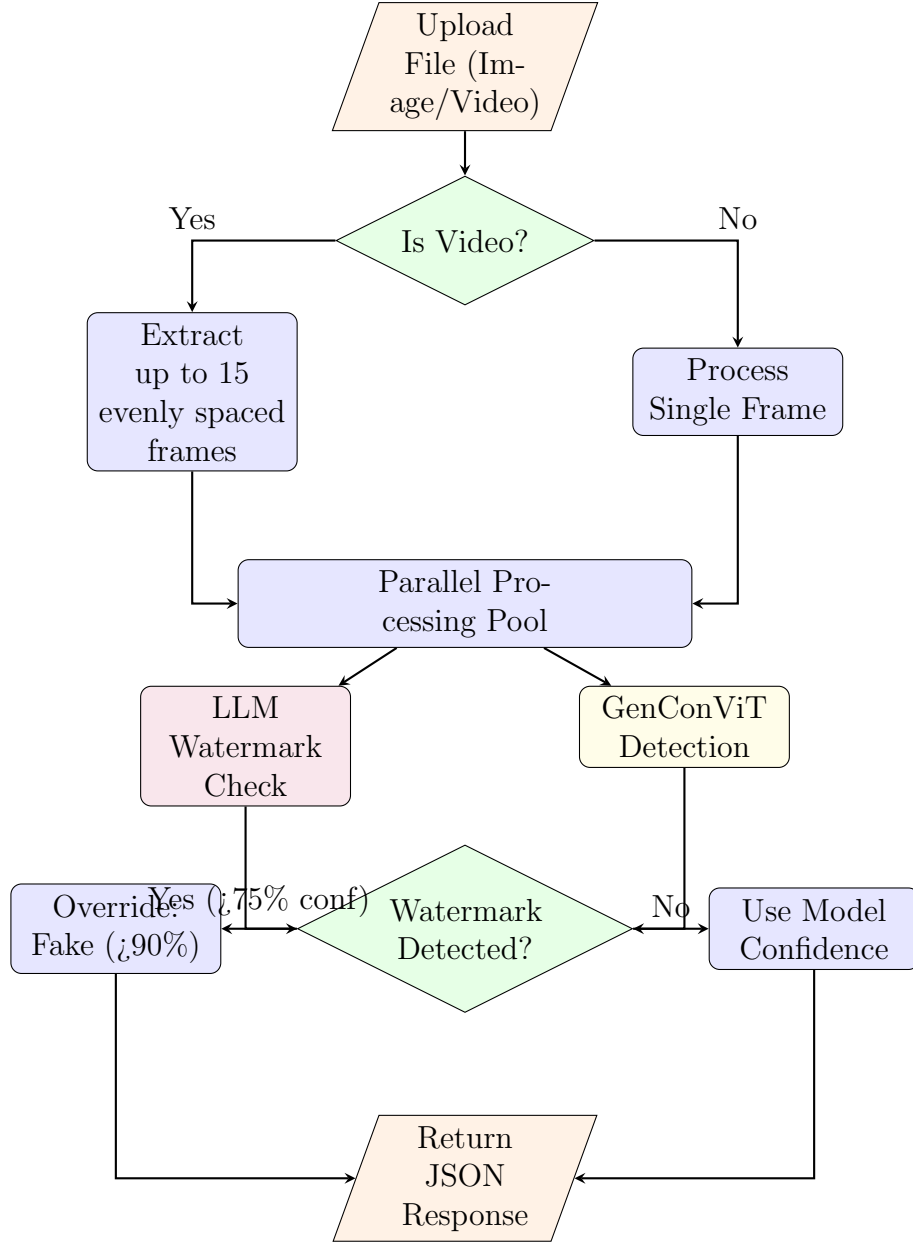


Figure 4: File Upload and Watermark Processing Pipeline

The FastAPI backend uses asynchronous parallel processing to analyze the uploaded media. If a video is provided, the backend systematically subsamples up to 15 frames to maintain low latency. An LLM-based watermark check is run in parallel with the main deep learning classification. If a synthesized watermark (e.g., from an AI generator) is strongly detected, it automatically overrides the pipeline to flag the content as a deepfake.

5.4 System Components

The system is divided into three major components:

5.4.1 Frontend (Android Capture System)

The frontend is responsible for:

- Capturing screen frames using the MediaProjection API
- Capturing internal device audio using AudioPlaybackCapture
- Performing lightweight preprocessing
- Sending data to the backend server
- Displaying results via a floating overlay interface

5.4.2 Backend (Deep Learning Models)

The backend performs the core inference tasks:

- Visual deepfake detection using image-based models
- Audio deepfake detection using speech models
- Multimodal fusion of predictions
- Returning classification results and confidence scores

5.4.3 Communication Layer

The frontend and backend communicate through REST APIs:

- Data is sent as multipart HTTP requests (image + audio)
- Responses are returned in JSON format
- Communication is asynchronous to avoid blocking the capture pipeline

5.5 Design Goals

The system was designed with the following key objectives:

- **Real-time operation:** Continuous detection while the user consumes media
- **Application independence:** Works across all apps without integration
- **Non-intrusive interface:** Overlay does not interrupt user experience
- **Multimodal robustness:** Combines audio and visual signals
- **Scalability:** Backend models can be updated independently

5.6 Key Challenges

During development, several challenges were encountered:

- Handling compressed and low-quality real-world media
- Maintaining low latency while processing continuous data streams
- Ensuring stable capture across different Android devices
- Designing models that generalize beyond training datasets

These challenges significantly influenced the choice of model architectures and system design decisions discussed in later sections.

6 Frontend System: Android Capture Pipeline

The frontend system is implemented as an Android application responsible for capturing media content in real time and interfacing with the backend detection models. The design focuses on continuous operation, minimal user interference, and compatibility across different applications.

6.1 Overview

The frontend operates as a foreground service that runs independently of the user interface. It continuously captures screen frames and audio data, preprocesses them, and sends them to the backend server for inference. The results are displayed using a floating overlay that remains visible across all applications.

6.2 Screen Capture

Screen capture is implemented using the Android `MediaProjection` API, which allows capturing the device display without requiring access to individual applications.

- Frames are captured at a rate of approximately **1 frame per second**
- A `VirtualDisplay` is created to mirror the device screen
- Frames are received through an `ImageReader`
- Raw RGBA pixel buffers are converted into `Bitmap` format
- Each frame is resized to **720 × 720** pixels
- Frames are compressed into JPEG format (95% quality) for efficient transmission

This approach ensures that content from any application (e.g., video players, social media apps) can be analyzed without requiring integration with those applications.

6.3 Audio Capture

Audio capture is implemented using the `AudioPlaybackCapture` API introduced in Android 10, which allows recording internal device audio.

- Audio is captured at **16 kHz**, mono, 16-bit PCM format
- Data is collected in **3-second segments**
- Each segment is packaged into a standard WAV file

If internal audio capture is restricted (e.g., due to app limitations), the system automatically falls back to microphone input to ensure continuous audio availability.

6.4 Data Transmission

Captured video frames and audio segments are sent to the backend server using HTTP requests.

- Data is transmitted using **multipart POST requests**
- Each request contains:
 - One image frame (JPEG)
 - One audio segment (WAV)
- Requests are handled asynchronously to avoid blocking the capture pipeline

This design enables simultaneous multimodal analysis on the backend.

6.4.1 WebSocket Streaming Implementation

To minimize latency and avoid blocking the Android UI thread, the frontend communicates with the backend via a continuous WebSocket stream. Data transmission is implemented using a custom binary protocol where video frames and audio chunks are prefixed with byte markers (0x01 and 0x02 respectively).

The following Kotlin snippet from `StreamingClient.kt` demonstrates the non-blocking, fire-and-forget architecture used for transmitting visual data:

```
// Fire-and-forget transmission (never blocks)
fun sendFrame(jpeg: ByteArray): Boolean {
    if (!isConnected || websocket == null) return false
    val buf = ByteArray(1 + jpeg.size)
    buf[0] = 0x01 // VIDEO_MARKER
    System.arraycopy(jpeg, 0, buf, 1, jpeg.size)
    val ok = websocket?.send(buf.toString()) ?: false
    if (ok) framesSent++
    return ok
}

fun sendAudio(wav: ByteArray): Boolean {
    if (!isConnected || websocket == null) return false
    val buf = ByteArray(1 + wav.size)
    buf[0] = 0x02 // AUDIO_MARKER
    System.arraycopy(wav, 0, buf, 1, wav.size)
    return websocket?.send(buf.toString()) ?: false
}
```

This design ensures that the capture service can stream data at the required framerate without waiting for HTTP handshakes on every frame.

6.5 Foreground Service Design

The capture pipeline runs as a **foreground service**, which provides the following advantages:

- Continuous operation even when the app is in the background
- Reduced likelihood of being terminated by the system
- Persistent notification to inform the user that capture is active

This is critical for maintaining real-time detection while the user switches between applications.

6.6 Floating Overlay Interface

The system provides feedback to the user through a floating overlay that appears on top of all applications.

- Displays system status: Idle, Capturing, Analyzing
- Shows prediction results: Real or Fake
- Displays confidence score
- Uses color indicators:
 - Green: Real
 - Red/Amber: Fake
- Can be dragged to any position on the screen

The overlay ensures that the detection results are visible without interrupting the user's workflow.

6.7 Implementation Architecture

The frontend is modular and consists of multiple components:

- **MainActivity:** Handles UI controls and permissions
- **CaptureService:** Manages the capture lifecycle
- **VideoFrameCapturer:** Handles screen frame extraction
- **AudioCapturer:** Handles audio recording
- **ApiClient:** Manages communication with backend
- **OverlayManager:** Controls the floating overlay

Each component has a clearly defined responsibility, allowing easier debugging and scalability.

6.8 Development and Operational Context

To achieve seamless screen capturing on mobile devices, careful handling of Android OS permissions was required.

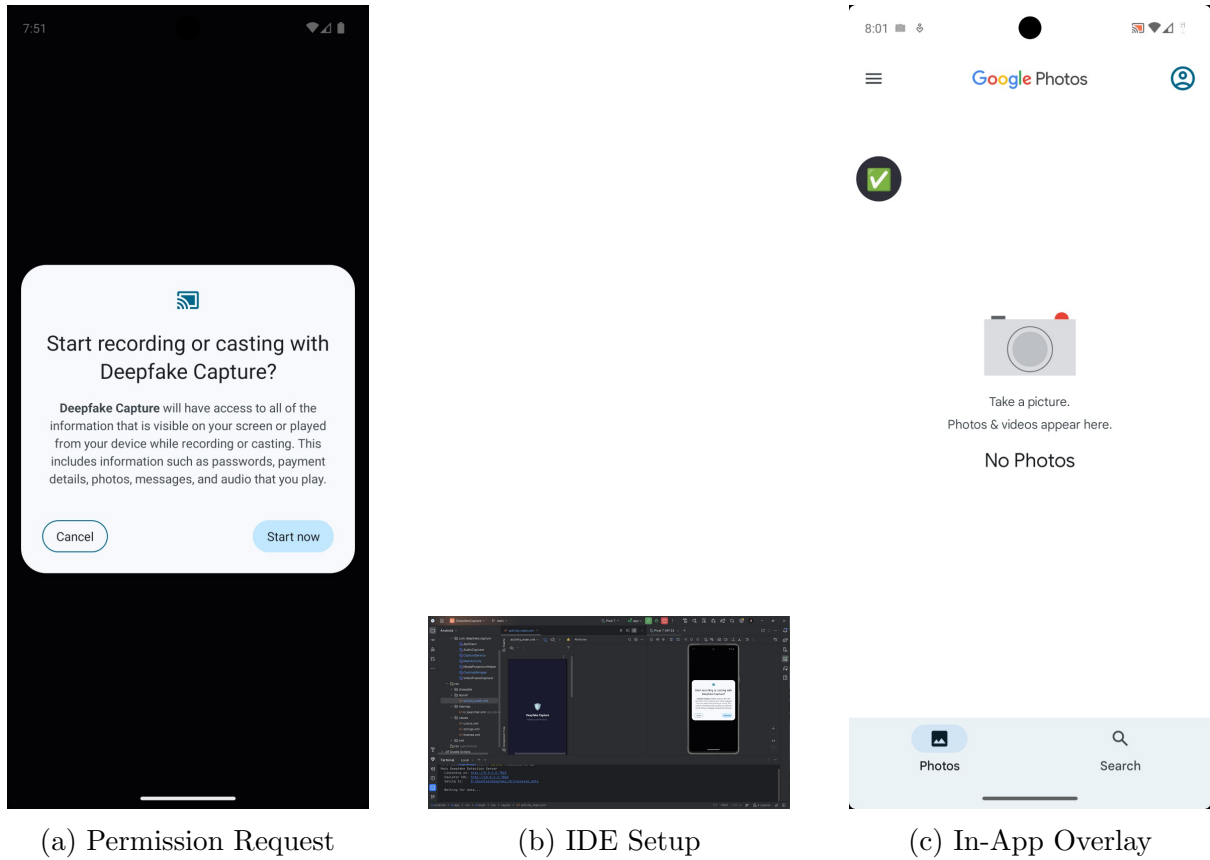


Figure 5: Frontend Permission Flows and Active Overlay

Simultaneously, the FastAPI server records continuous telemetry regarding inference latencies, batch processing, and multimodal fusion adjustments.

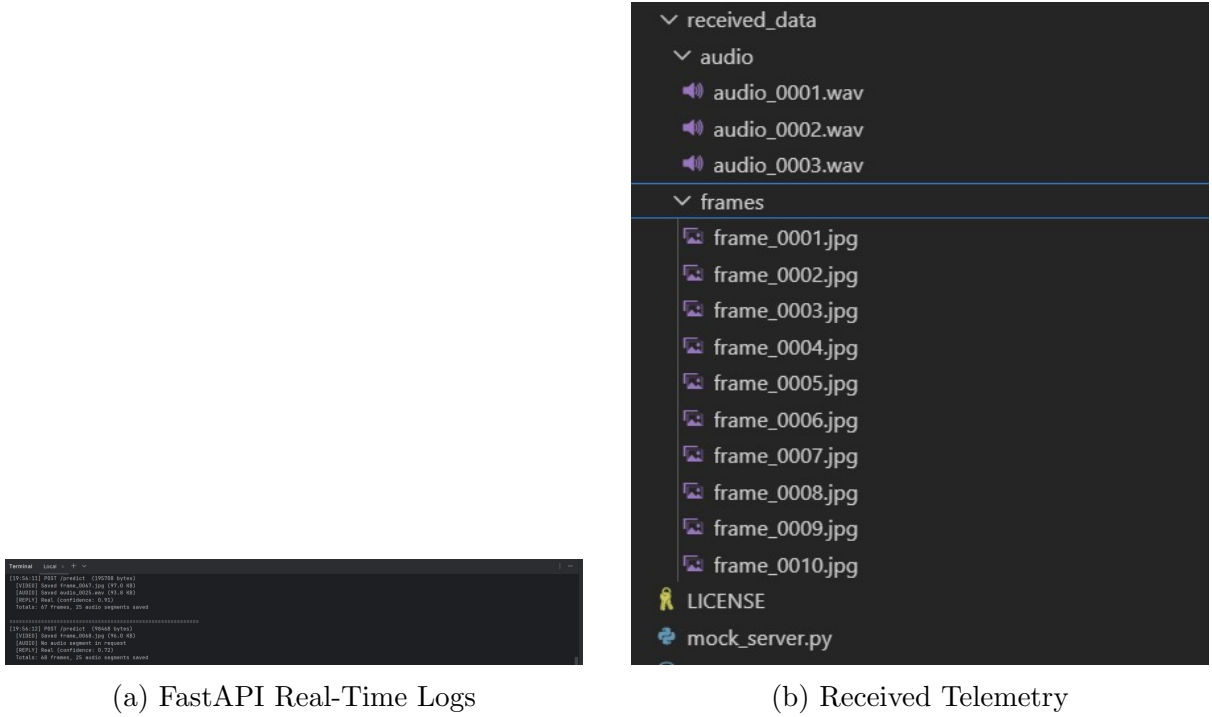


Figure 6: Backend Operations and Telemetry

6.9 Challenges in Frontend Design

Several challenges were encountered during implementation:

- Ensuring stable screen capture across different devices
- Handling restrictions on internal audio capture for certain apps
- Maintaining low latency while continuously sending data
- Avoiding excessive battery consumption
- Managing synchronization between audio and video streams

These constraints influenced decisions such as reducing frame rate and batching audio data.

6.10 System Architecture and Real-Time Detection Flow

The real-time mobile detection system operates as a background root service that intercepts decoded video frames from the Android media pipeline. This allows it to work transparently alongside existing video applications without interrupting video playback.

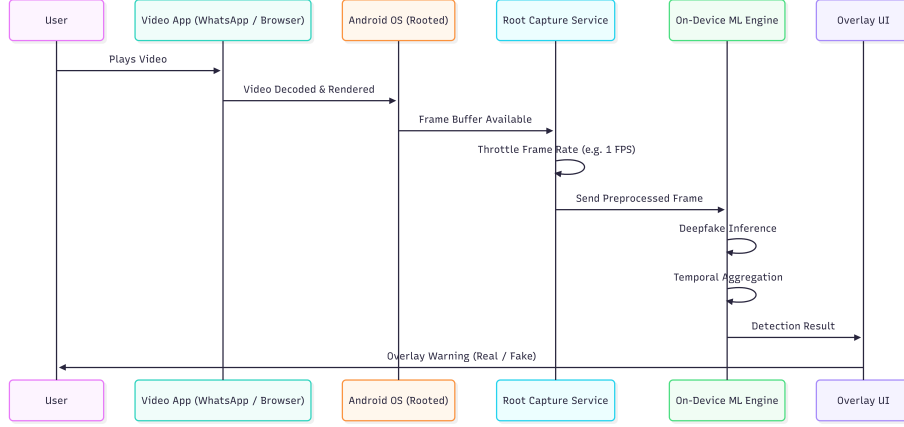


Figure 7: Real-Time Mobile Detection Flow

The detection results are communicated via a floating overlay UI.

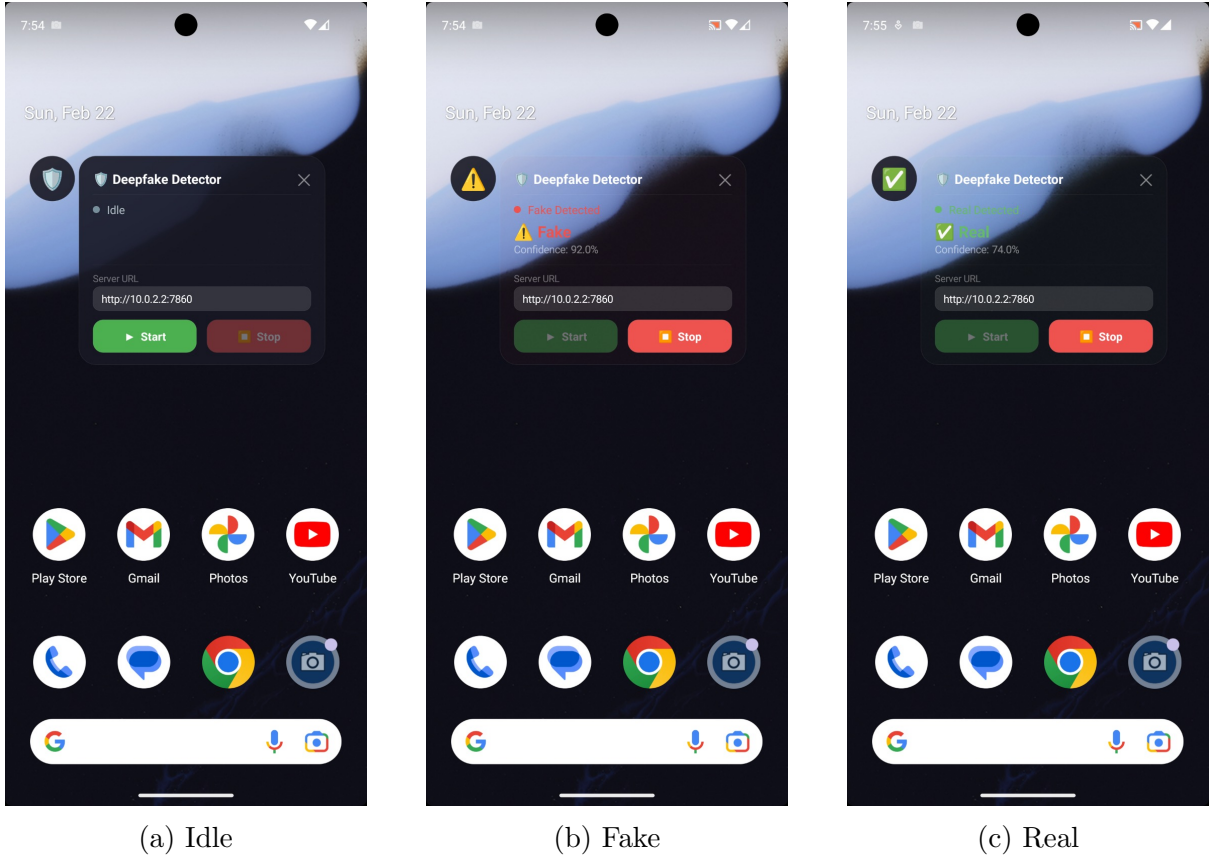


Figure 8: Overlay States during operation

6.11 Threat Model and Security Considerations

The proposed system operates at the operating system level on rooted Android devices. The primary threats include unauthorized frame access, malicious model manipulation, false detection attacks, and user privacy leakage. To mitigate these threats, frame data is processed in-memory, the detection model is embedded, and no inference results are transmitted off-device.

7 Initial Approaches and Observed Limitations

During the early stages of development, multiple deep learning architectures were explored for visual deepfake detection. These included convolutional neural networks (CNNs) and Vision Transformer (ViT)-based models. While these approaches showed reasonable performance on controlled datasets, they did not perform reliably in real-world conditions.

This section outlines the architectures that were tried and the key reasons for their limitations.

7.1 CNN-Based Approaches

Initial experiments were conducted using standard convolutional neural networks for binary classification (real vs fake). These models were trained on labeled datasets containing real and manipulated images.

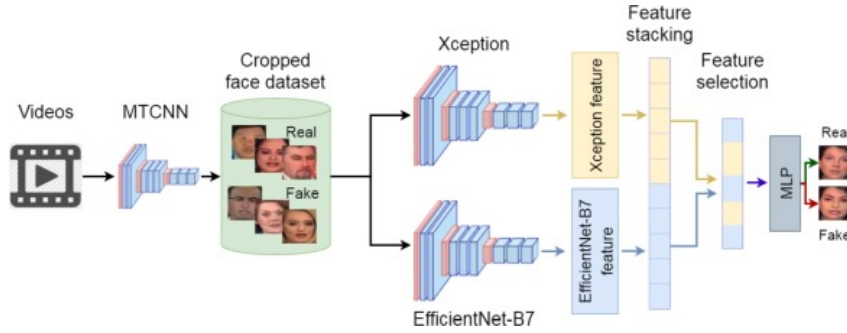


Figure 9: CNN-based XceptionNet model architecture

7.1.1 Architecture Overview

Typical CNN-based pipelines followed this structure:

- Input image resized to a fixed resolution
- Convolutional layers for feature extraction
- Pooling layers for spatial reduction
- Fully connected layers for classification

Pretrained models such as ResNet and EfficientNet variants were also explored for transfer learning.

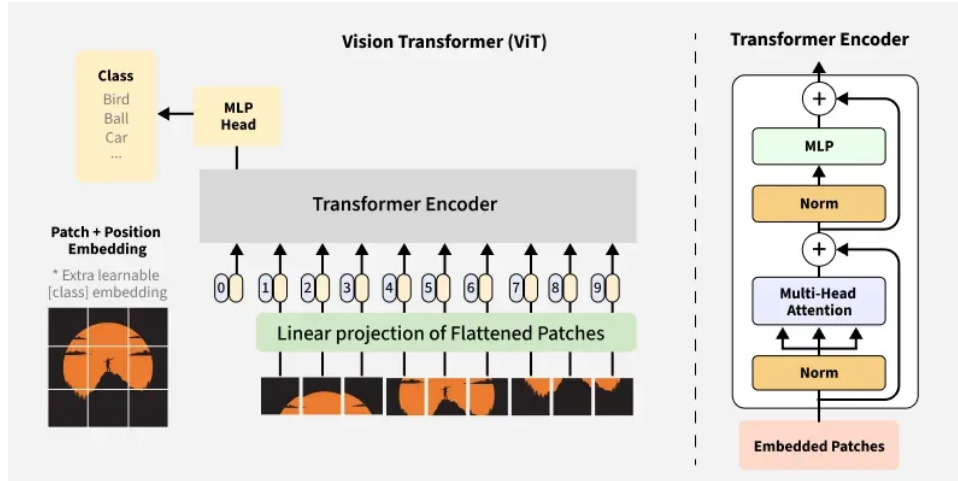


Figure 10: Vision Transformer based Model Architecture

7.1.2 Observed Performance

CNN models performed reasonably well on validation datasets and achieved good accuracy under controlled conditions. However, their performance degraded significantly in real-world scenarios.

7.1.3 Limitations

The main issues observed were:

- **Overfitting to training data:** Models learned dataset-specific artifacts rather than general deepfake features
- **Sensitivity to compression:** Performance dropped on compressed videos and low-resolution frames
- **Poor generalization:** Models failed on unseen deepfake generation techniques
- **Limited context understanding:** CNNs primarily capture local patterns and struggle with global inconsistencies

These limitations made CNN-based approaches unsuitable for deployment in a real-time system where input data is highly variable.

7.2 Vision Transformer (ViT) Approaches

To address the limitations of CNNs in capturing global context, Vision Transformer-based models were explored.

7.2.1 Architecture Overview

ViT models process images as sequences of patches:

- Input image divided into fixed-size patches
- Each patch embedded into a vector representation

- Transformer encoder used to model relationships between patches
- Classification head predicts real vs fake

7.2.2 Observed Performance

ViT-based models showed improved ability to capture global relationships in images compared to CNNs. However, their real-world performance was still inconsistent.

7.2.3 Limitations

The key issues identified were:

- **Data dependency:** Requires large and diverse datasets to generalize well
- **Sensitivity to noise:** Performance degraded on noisy or compressed inputs
- **Lack of artifact-specific learning:** Did not reliably detect subtle manipulation artifacts
- **Instability in real-world inputs:** Predictions fluctuated significantly across frames

7.2.4 SigLIP Alternative Architecture

As an alternative to standard ViT models, the SigLIP (Sigmoid Loss for Language Image Pre-Training) architecture was also evaluated. SigLIP replaces the standard softmax-based contrastive loss with a pairwise sigmoid loss, which can improve zero-shot classification and robustness across diverse image sources.

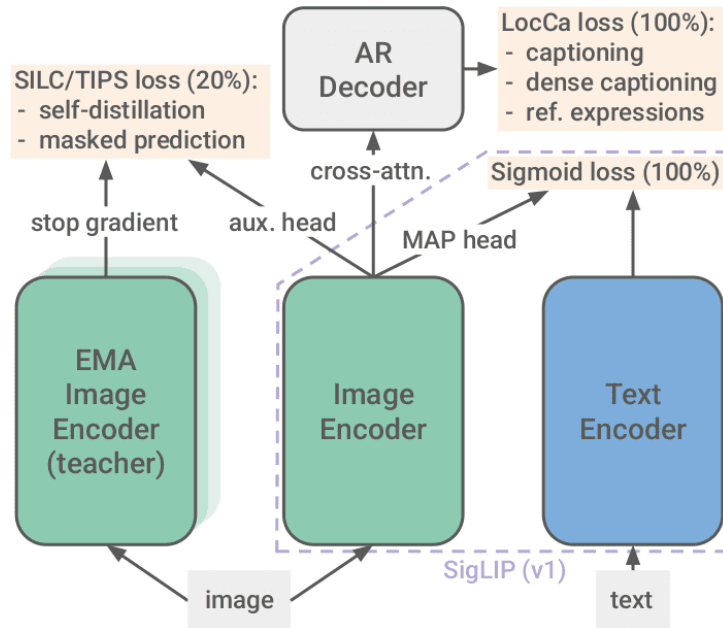


Figure 11: SigLIP Encoder Architecture

7.3 Key Observations

From these experiments, several important insights were obtained:

- High validation accuracy does not translate to real-world performance
- Models trained purely as classifiers struggle with unseen manipulations
- Both CNN and ViT approaches rely heavily on surface-level features
- Real-world media introduces variability (compression, scaling, noise) not captured during training

These observations highlighted the need for models that go beyond direct classification and incorporate deeper structural understanding of images.

7.4 Conclusion from Initial Attempts

The initial approaches demonstrated that standard classification-based models are insufficient for robust deepfake detection in real-time environments. This led to the exploration of more advanced architectures that incorporate generative modeling, latent representations, and multi-scale feature extraction, which are discussed in the next section.

8 Advanced Visual Models: Hybrid Generative–Transformer Approach

After observing the limitations of standard CNN and Vision Transformer models, more advanced architectures were explored that incorporate generative modeling and multi-scale feature extraction. These models aim to capture not only surface-level artifacts but also the underlying structure and distribution of real versus manipulated images.

One such architecture that showed significantly improved performance is a hybrid model combining autoencoders, variational autoencoders, convolutional networks, and transformer-based components.

8.1 Motivation

The failure of earlier models indicated that classification-based approaches alone were insufficient. Specifically:

- Models were learning *what fake looks like* instead of *what real should look like*
- They failed on unseen manipulations and real-world data distributions
- They lacked robustness to compression and noise

To address this, the focus shifted toward models that learn:

- Structural consistency of real images
- Latent distributions of facial features
- Multi-scale spatial and contextual relationships

8.2 Architecture Overview

The advanced model consists of two parallel processing branches:

- **Autoencoder branch (AE)**
- **Variational Autoencoder branch (VAE)**

Both branches reconstruct the input image and extract features, which are further processed using convolutional and transformer-based components.

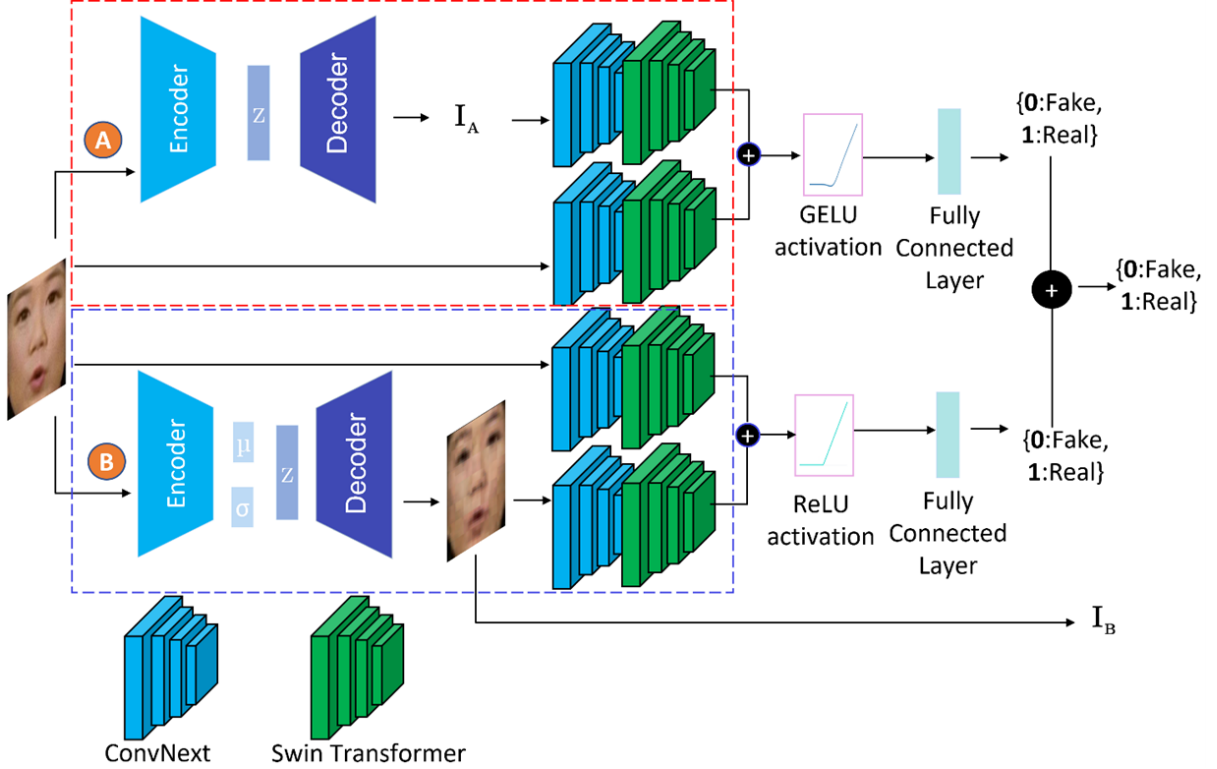


Figure 12: Architecture of the GenConViT Hybrid Model

8.2.1 Autoencoder Branch

The autoencoder branch consists of:

- **Encoder:** compresses input image into a latent representation
- **Decoder:** reconstructs the image from the latent space

This branch focuses on learning the structural patterns of real images. Reconstruction errors implicitly capture inconsistencies introduced by deepfake generation.

8.2.2 Variational Autoencoder Branch

The VAE branch extends the autoencoder by modeling a probabilistic latent space:

- **Encoder** outputs mean (μ) and variance (σ)

- Latent vector is sampled using reparameterization
- Decoder reconstructs the image from sampled latent representation

This enables the model to learn the distribution of real images and detect deviations caused by synthetic manipulations.

8.2.3 Feature Extraction Layers

Outputs from both branches are passed through:

- **ConvNeXt:** captures local features such as texture inconsistencies, blending artifacts, and edge distortions
- **Swin Transformer:** captures global relationships such as facial symmetry, alignment, and structural coherence

The combination allows the model to analyze both fine-grained artifacts and high-level inconsistencies.

8.2.4 Fusion and Classification

For each branch:

- Features from ConvNeXt and Swin Transformer are combined
- Non-linear activations (e.g., GELU or ReLU) are applied
- Fully connected layers produce classification outputs

Final predictions from both branches are aggregated to produce the final classification (real or fake).

8.3 Why This Approach Worked

This architecture addressed the key limitations observed in earlier models:

- **Latent space learning:** By reconstructing images, the model learns what real images should look like, rather than only detecting known fake patterns
- **Distribution modeling:** The VAE captures variability in real data, improving generalization
- **Multi-scale analysis:** Combining convolutional and transformer layers enables both local and global feature extraction
- **Robustness to real-world data:** The model performs better on compressed and noisy inputs compared to pure classifiers

8.4 Observed Improvements

Compared to earlier approaches, the hybrid model demonstrated:

- More stable predictions across frames
- Improved performance on unseen deepfake techniques
- Better robustness to compression artifacts
- Reduced false positives on real-world media

8.5 Limitations

Despite improved performance, this approach has several drawbacks:

- **High computational cost:** The model is significantly heavier than standard CNN or ViT models
- **Training complexity:** Balancing reconstruction and classification objectives is non-trivial
- **Not suitable for real-time mobile inference:** Requires backend processing

8.6 Conclusion

The hybrid generative–transformer architecture represents a significant improvement over traditional classification-based models for deepfake detection. By combining reconstruction-based learning with multi-scale feature extraction, the model achieves better generalization and robustness in real-world scenarios.

This model forms the basis of the visual detection component used in the backend system.

8.7 Training and Evaluation Details

The model is built upon the ViT-Base (Patch16-224) architecture, pre-trained on ImageNet-21k. It uses the AdamW optimizer with a learning rate of 1×10^{-6} . Classification performance on the validation set is detailed below:

Class	Precision	Recall	F1-score	Support
Real	0.9683	0.8708	0.9170	28,001
Fake	0.8826	0.9715	0.9249	28,000
Accuracy	0.9212			

Table 1: Classification performance

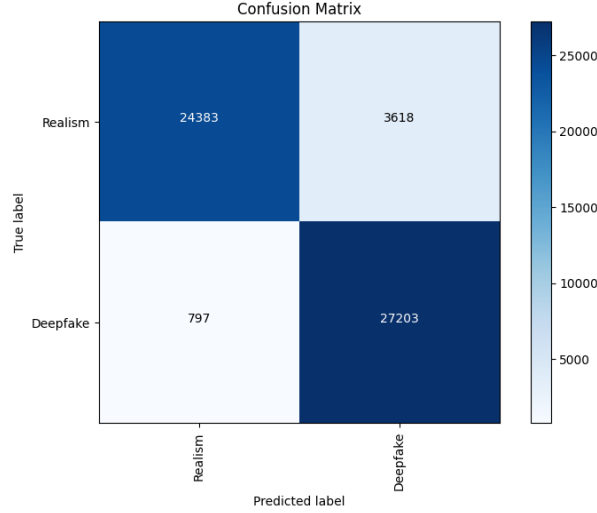


Figure 13: Confusion Matrix

9 Audio Deepfake Detection Model

In addition to visual analysis, an audio-based deepfake detection model was developed to improve overall system robustness. Audio signals provide complementary information that is often harder to manipulate perfectly compared to visual content, making them a critical component in multimodal detection.

9.1 Motivation

Visual-only models were found to be insufficient in several scenarios:

- High-quality deepfakes with minimal visual artifacts
- Low-resolution or heavily compressed video
- Partial occlusion or non-frontal faces

Audio, on the other hand, often contains subtle inconsistencies in speech patterns, prosody, and signal characteristics that can be leveraged for detection. This motivated the development of a dedicated audio detection pipeline.

9.2 Model Architecture

The audio model is based on the **Wav2Vec2** architecture, a self-supervised speech representation model designed to learn meaningful features directly from raw audio.

9.2.1 Architecture Overview

- Input: Raw audio waveform (16 kHz, mono)
- Feature Encoder: Converts waveform into latent speech representations
- Transformer Layers: Capture temporal dependencies and contextual information

- Classification Head: Outputs probability of real vs fake audio

Wav2vec 2.0 Pre-training

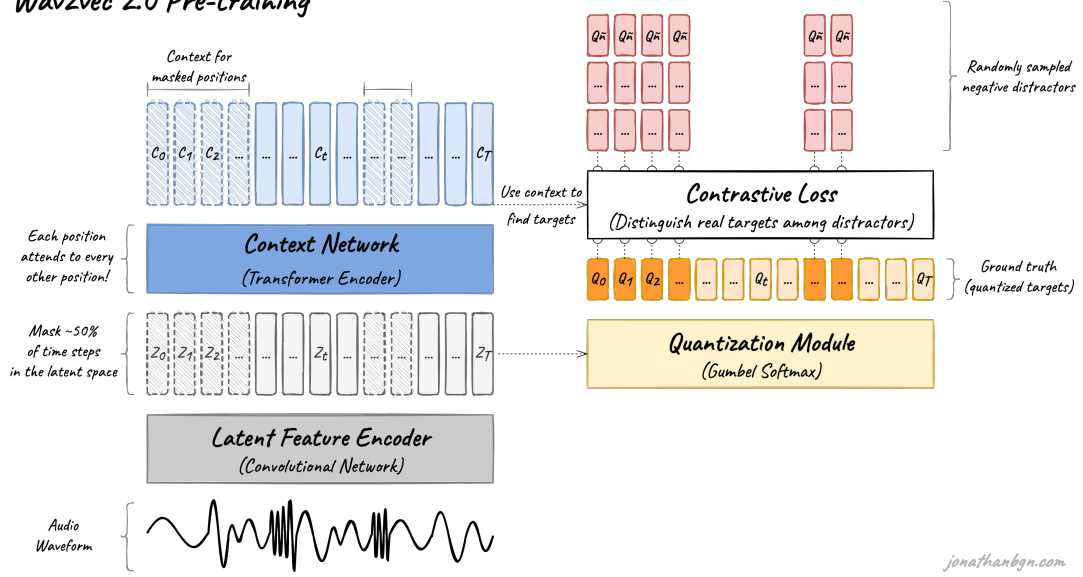


Figure 14: Architecture of the Wav2Vec2 Audio Deepfake Detection Model

The model was fine-tuned through multiple stages:

- Base model: wav2vec2-base
- Intermediate fine-tuning on speech datasets
- Final fine-tuning on deepfake audio datasets

The audio model is built upon the Wav2Vec2-base architecture, following a multi-stage fine-tuning pipeline (mo-thecreator/wav2vec2-base-finetuned to MelodyMachine/Deepfake-audio-). The final model contains approximately 94.6 million parameters. Training was performed with a learning rate of 3×10^{-5} , Adam optimizer, and cosine scheduler.

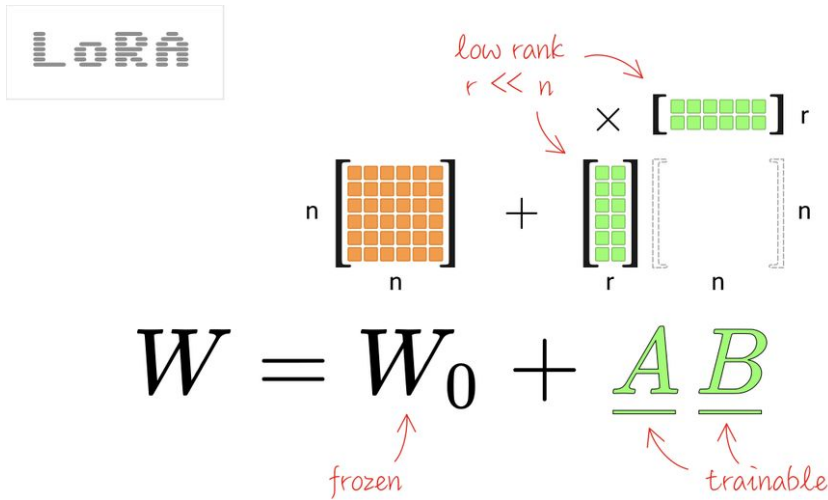


Figure 15: LoRA Fine-Tuning Strategy for the Wav2Vec2 Model

Table 2: Training Results for Audio Model

Epoch	Train Loss	Val Loss	Accuracy
1	0.0503	0.0514	0.9858
5	0.0115	0.0141	0.9973

9.3 Training Configuration

The model was trained using the following setup:

- Task: Binary classification (Real vs Fake)
- Optimizer: Adam
- Learning rate: 3×10^{-5}
- Batch size: 32 (effective batch size increased using gradient accumulation)
- Number of epochs: 5

Audio data consisted of:

- Real human speech recordings
- Synthetic speech generated using voice cloning and text-to-speech systems

9.4 Performance

The audio model demonstrated very strong performance:

- Final accuracy: **99.7%**
- Low validation loss
- Consistent performance across different audio samples

Compared to visual models, the audio model was significantly more stable and reliable.

9.5 Why the Audio Model Worked Well

Several factors contributed to the strong performance:

- **Rich temporal features:** Speech contains structured temporal patterns that are difficult to replicate perfectly
- **Self-supervised learning:** Wav2Vec2 learns robust representations from raw audio without relying on handcrafted features
- **Less sensitivity to visual noise:** Unlike images, audio is not affected by compression artifacts in the same way
- **Clear discriminative cues:** Synthetic audio often contains subtle artifacts in pitch, timing, and waveform consistency

9.6 Limitations

Despite strong performance, the audio model has certain limitations:

- **Dependency on audio availability:** Cannot operate when audio is absent or muted
- **Background noise sensitivity:** Performance may degrade in noisy environments
- **Short segment analysis:** Using 3-second segments may miss long-term inconsistencies

9.7 Role in the Overall System

The audio model plays a critical role in the multimodal system:

- Acts as a strong complementary signal to visual detection
- Helps reduce false positives from visual models
- Improves detection in cases where visual artifacts are minimal

In many real-world scenarios, the audio model provides higher confidence predictions than the visual model, making it an essential component of the overall system.

10 Multimodal Fusion Strategy

To improve robustness and reliability, the system combines predictions from both visual and audio models. This multimodal approach helps address the limitations of individual modalities and provides more consistent performance in real-world scenarios.

10.1 Motivation

Single-modality approaches were found to be insufficient:

- Visual models struggled with high-quality deepfakes and compressed inputs
- Audio models could not operate when audio was unavailable or noisy

These observations motivated the integration of both modalities to leverage their complementary strengths.

10.2 Fusion Approach

The system uses a **late fusion strategy**, where audio and visual inputs are processed independently, and their outputs are combined at the decision level.

10.2.1 Pipeline

1. Video frames are processed by the visual model
2. Audio segments are processed by the audio model
3. Each model produces:
 - A classification label (Real or Fake)
 - A confidence score
4. The outputs are combined to produce a final prediction

10.3 Fusion Logic

The fusion strategy is based on confidence-aware decision rules:

- If both models agree, the prediction is accepted with high confidence
- If one model has significantly higher confidence, it is given higher weight
- If both models are uncertain, the system marks the result as low confidence

This approach avoids over-reliance on any single modality.

10.4 Handling Missing Modalities

The system is designed to handle cases where one modality is unavailable:

- If audio is unavailable, the system falls back to visual-only prediction
- If visual input is unreliable, audio predictions are prioritized

This ensures continuous operation under varying conditions.

10.5 Advantages of Multimodal Fusion

The multimodal approach provides several benefits:

- **Improved robustness:** Combines complementary signals
- **Reduced false positives:** Errors in one modality can be corrected by the other
- **Better generalization:** Handles diverse real-world scenarios
- **Higher confidence predictions:** Agreement between modalities increases reliability

10.6 Observed Behavior

In practice, the following patterns were observed:

- Audio model often provides stable predictions across time
- Visual model may fluctuate depending on frame quality
- Combined predictions are more consistent than individual outputs

10.7 Limitations

Despite improvements, some limitations remain:

- Simple late fusion does not fully capture cross-modal relationships
- Temporal alignment between audio and video is not explicitly modeled
- Conflicting predictions can still occur in edge cases

10.8 Future Improvements

More advanced fusion strategies can be explored in future work:

- Feature-level fusion instead of decision-level fusion
- Temporal aggregation across multiple frames and audio segments
- Cross-modal attention mechanisms

10.9 Conclusion

The multimodal fusion strategy significantly improves the reliability of the system by combining the strengths of both audio and visual models. While the current approach is relatively simple, it provides a strong foundation for further improvements.

10.10 Adaptive Reinforcement Learning Fusion Implementation

The fusion of audio and visual confidence scores is handled dynamically using a gradient-based reinforcement learning approach. The system adjusts the weights of the visual and audio models based on user feedback (corrections).

The following Python snippet from `rl_system.py` illustrates the weight update logic when the system receives a ground-truth correction:

```
def update_from_feedback(self, prediction, user_label, user_confidence=1.0):
    was_correct = (prediction['verdict'] == user_label)

    if not was_correct and 'video_fake_score' in prediction:
        video_score = prediction['video_fake_score']
        audio_score = prediction['audio_fake_score']
```

```

user_fake_prob = 0.75 if user_label in ['FAKE', 'SUSPICIOUS'] else 0.25
video_error = abs(video_score - user_fake_prob)
audio_error = abs(audio_score - user_fake_prob)

# Adjust weights: increase weight of more accurate modality
adjustment = self.learning_rate * user_confidence
if video_error < audio_error:
    self.video_weight = np.clip(self.video_weight + adjustment, 0.5, 0.95)
else:
    self.video_weight = np.clip(self.video_weight - adjustment, 0.5, 0.95)

self.audio_weight = 1.0 - self.video_weight

```

This mechanism allows the system to continuously adapt its fusion strategy to real-world edge cases without requiring full model retraining.

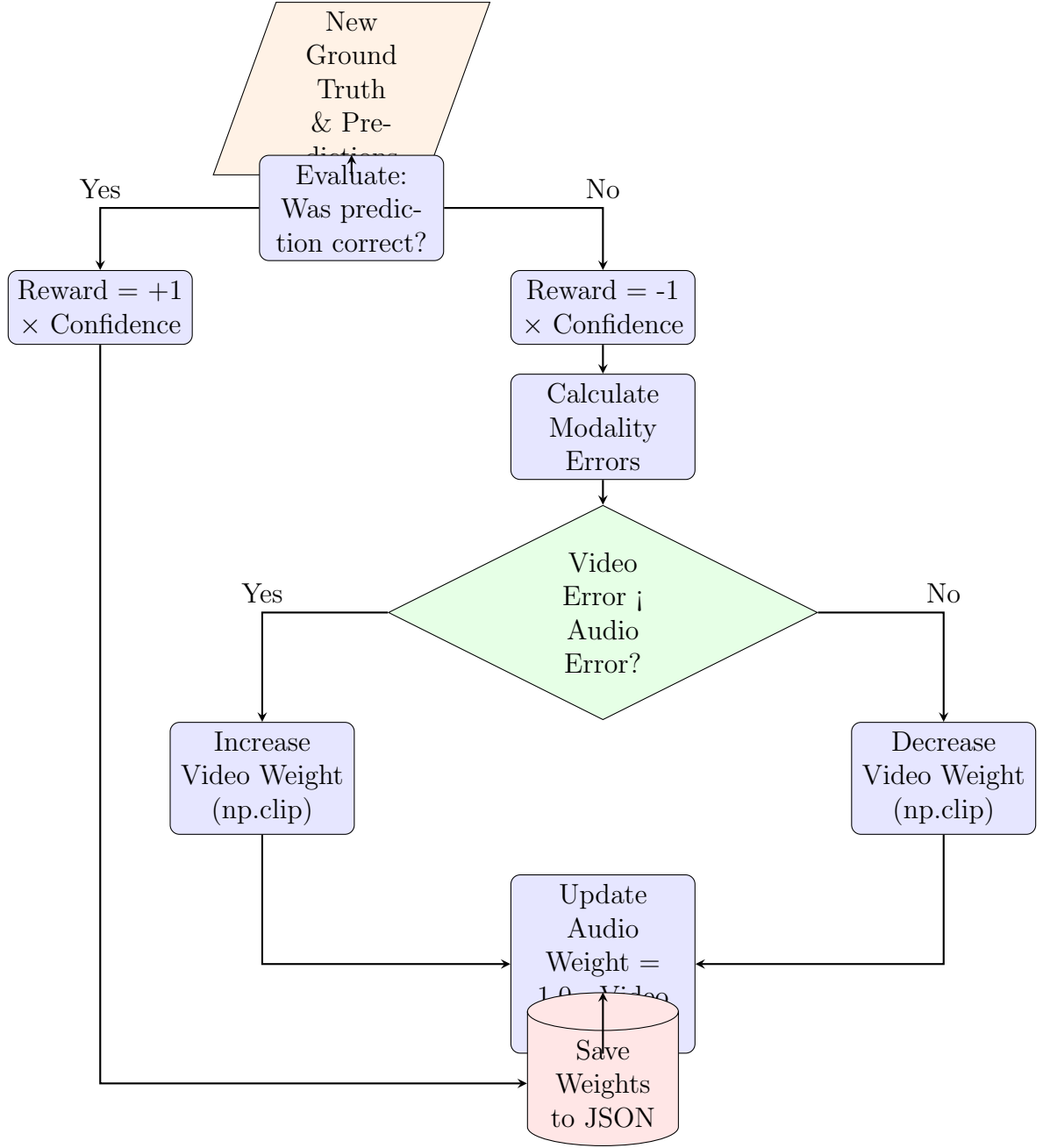


Figure 16: Reinforcement Learning Fusion Weight Update Logic

11 Exploration of LLM-Based and Proprietary Detection Approaches

In addition to traditional deep learning models, recent advances in multimodal large language models (LLMs) and proprietary detection systems were explored as potential solutions for deepfake detection. This includes models such as Gemini, GPT-based vision models, and watermarking-based systems such as SynthID.

While these approaches show promise in research settings, they were ultimately not used in the final system due to several practical and technical limitations.

11.1 Multimodal LLM-Based Detection

Recent multimodal LLMs are capable of processing images and generating reasoning-based outputs. These models can analyze visual content and provide explanations for whether an image appears real or manipulated.

11.1.1 Approach

The general approach involves:

- Providing an image as input to the model
- Prompting the model to classify the image as real or fake
- Extracting a confidence score or textual reasoning

11.1.2 Observations

During experimentation, several key behaviors were observed:

- Models often relied on high-level semantic cues such as lighting consistency, facial symmetry, and texture smoothness
- Outputs were sometimes inconsistent across repeated runs on the same input
- Confidence scores were not always reliable or well-calibrated
- Performance varied significantly depending on prompt design

11.1.3 Limitations

The primary limitations of LLM-based approaches were:

- **Inconsistent predictions:** Outputs varied across runs and were sensitive to prompt wording
- **Lack of determinism:** Difficult to guarantee stable behavior in a real-time system
- **Latency:** Inference time is too high for continuous real-time processing
- **Cost:** API-based models incur significant computational and financial overhead
- **Limited specialization:** Models are not specifically trained for deepfake detection tasks

These issues made LLM-based approaches unsuitable for deployment in a real-time detection pipeline.

11.2 Gemini-Based Approaches

Google’s Gemini models were evaluated as part of the exploration of multimodal reasoning systems.

11.2.1 Observations

- Predictions showed variability and occasional contradictions
- The model sometimes relied on contextual assumptions rather than visual evidence
- Performance did not consistently exceed that of specialized deep learning models

11.2.2 Conclusion

Despite strong general capabilities, Gemini was not suitable for this application due to lack of reliability and consistency in a production-like setting.

11.3 SynthID and Watermark-Based Detection

SynthID is a watermarking-based approach designed to embed imperceptible signals into generated content, allowing for later detection.

11.3.1 Approach

- Watermarks are embedded during content generation
- Detection involves identifying these embedded signals

11.3.2 Limitations

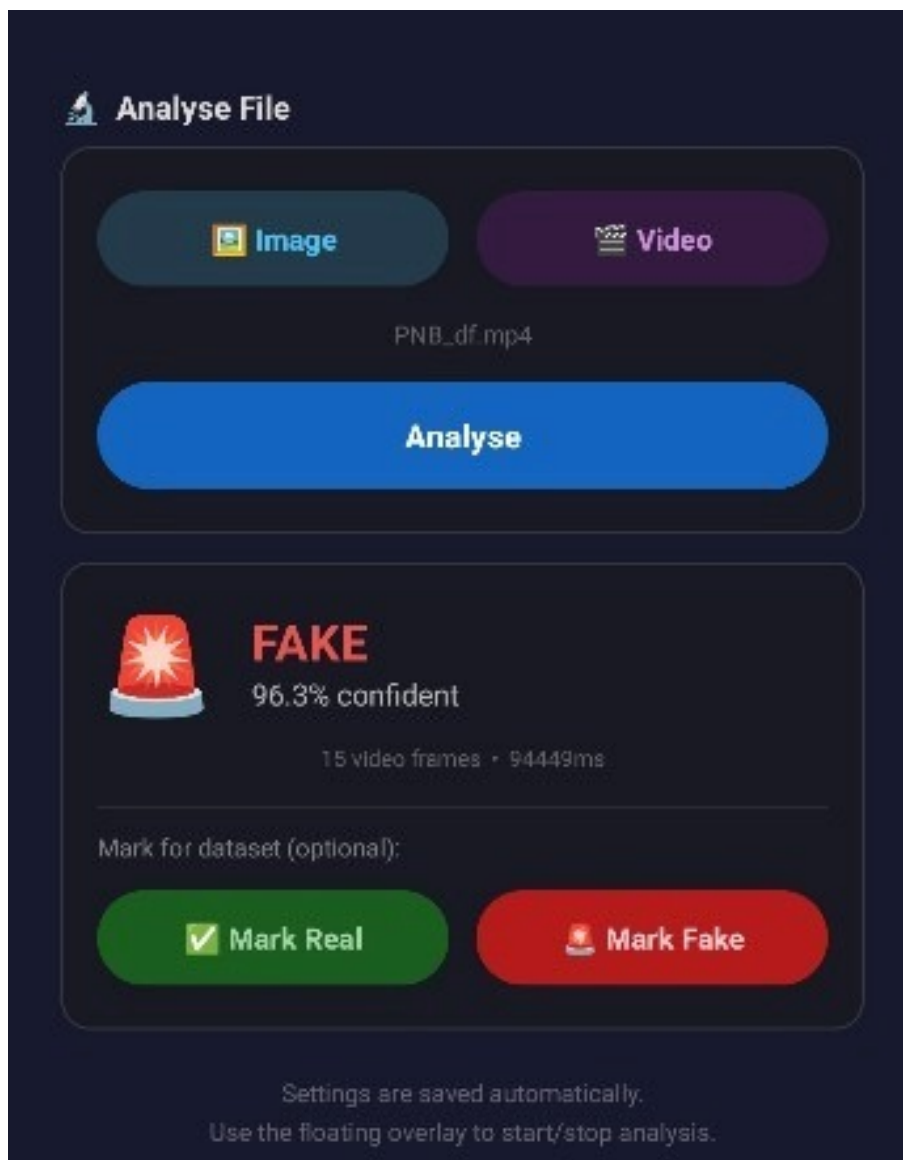
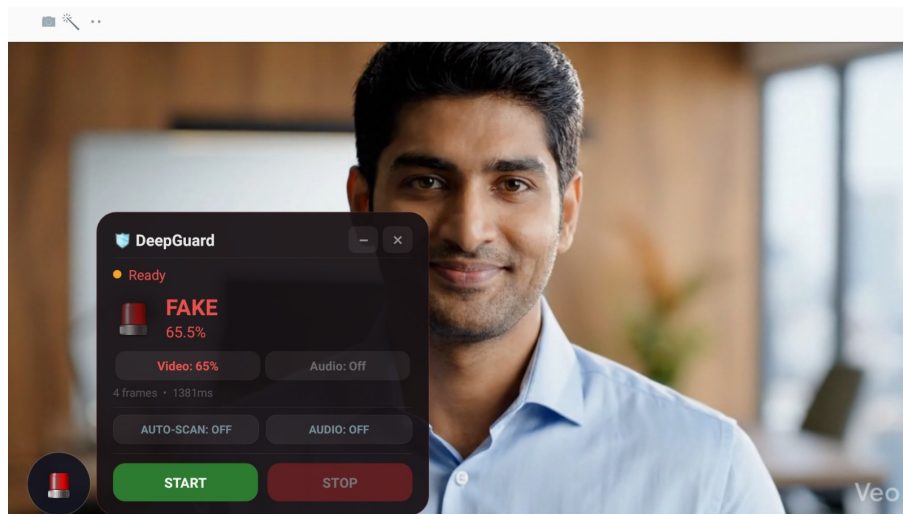
- **Requires cooperation from generation models:** Only works if the content was generated using supported systems
- **Not applicable to existing deepfakes:** Cannot detect content generated without watermarking
- **Limited real-world coverage:** Many deepfake tools do not implement such mechanisms

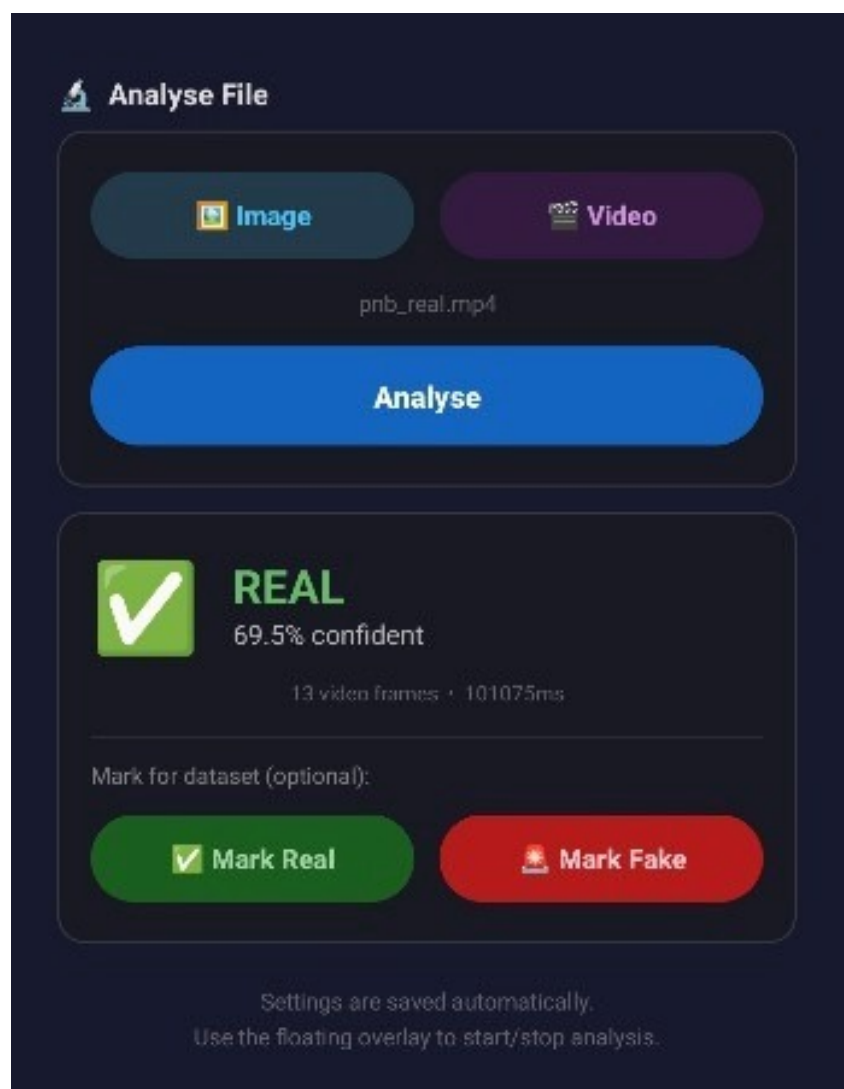
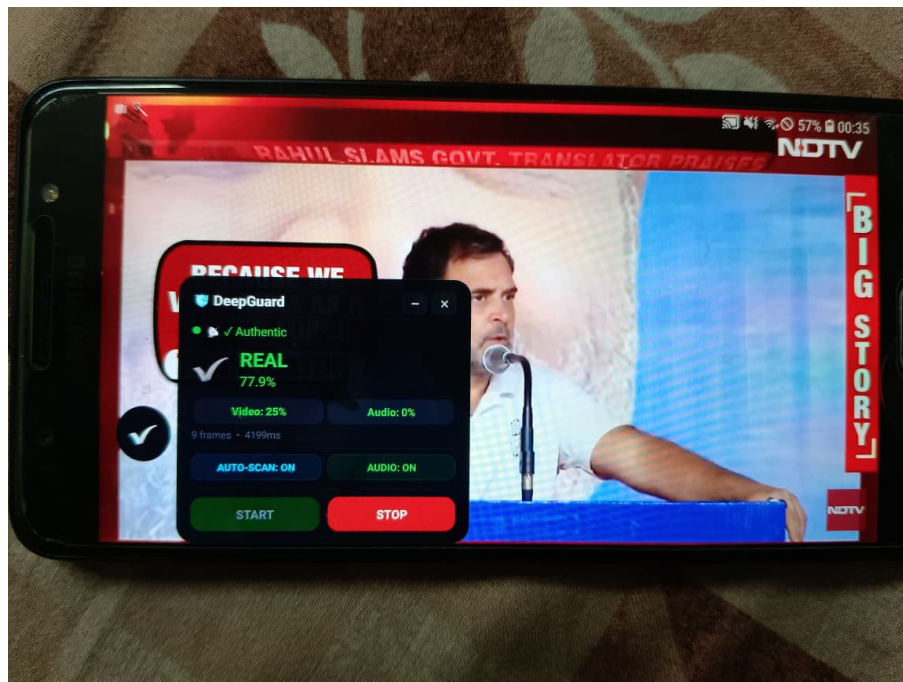
11.4 Comparison with Traditional Models

Compared to LLM-based and watermark-based approaches, traditional deep learning models offered:

- Deterministic outputs
- Lower latency
- Better suitability for real-time pipelines
- More direct control over training and optimization

11.5 Demo Results





11.6 Conclusion

While LLM-based and proprietary approaches represent an important direction for future research, they are currently not well-suited for real-time, system-level deepfake detection. The final system therefore relies on specialized audio and visual models that provide more stable and predictable performance.

12 Final System Design

The final system integrates the frontend capture pipeline, backend multimodal models, and a feedback-driven adaptive learning mechanism. The design focuses on real-time operation, robustness, and continuous improvement through user interaction.

12.1 End-to-End Pipeline

The complete system operates as follows:

1. Screen frames and audio are captured in real time on the Android device
2. Data is preprocessed into standardized formats (JPEG images and WAV audio)
3. Inputs are sent to the backend server via REST API
4. Visual and audio models perform independent inference
5. Predictions are combined using a multimodal fusion strategy
6. Results are displayed to the user through a floating overlay

12.2 System Architecture

The system is divided into three layers:

- **Capture Layer:** Handles real-time screen and audio acquisition
- **Inference Layer:** Runs deep learning models for detection
- **Interaction Layer:** Displays results and collects user feedback

This modular design allows independent updates to models and frontend components.

12.3 Adaptive Learning Mechanism

To improve performance over time and adapt to real-world scenarios, the system incorporates an **adaptive learning framework** that leverages user feedback.

12.3.1 Motivation

Static models trained on fixed datasets often fail to generalize to new deepfake generation techniques and real-world variations. To address this, the system allows continuous learning from user-provided ground truth.

12.3.2 User Feedback Integration

The system provides an interface for users to:

- Mark predictions as correct or incorrect
- Provide ground truth labels (Real or Fake)

This feedback is logged and stored for further processing.

12.3.3 Reinforcement Learning Component

The adaptive learning mechanism incorporates reinforcement learning (RL)-inspired updates:

- Predictions are treated as actions
- User feedback acts as a reward signal:
 - Positive reward for correct predictions
 - Negative reward for incorrect predictions
- The system updates model parameters or fusion weights based on feedback trends

Rather than directly retraining the full model in real time, the system focuses on:

- Adjusting confidence thresholds
- Re-weighting audio vs visual contributions
- Prioritizing historically reliable modalities

12.3.4 Online Learning Strategy

Feedback data is accumulated and used in periodic updates:

- Batch retraining or fine-tuning on collected samples
- Updating decision boundaries based on real-world inputs
- Improving robustness to unseen manipulation techniques

12.3.5 Advantages

- **Continuous improvement:** System evolves with new data
- **Personalization:** Adapts to user-specific usage patterns
- **Better generalization:** Learns from real-world edge cases

12.3.6 Limitations

- Requires reliable user feedback
- Risk of noisy or incorrect labels
- Additional complexity in managing updates

12.4 Deployment Considerations

- Backend models handle heavy computation
- Frontend remains lightweight for real-time performance
- Communication latency is minimized through asynchronous requests

12.5 Conclusion

The final system combines real-time multimodal detection with adaptive learning capabilities, enabling both immediate feedback and long-term improvement. This approach provides a practical and scalable solution for deepfake detection in real-world environments.

13 Conclusion and Future Work

13.1 Conclusion

This project presents a real-time multimodal deepfake detection system designed for practical deployment on mobile devices. Unlike traditional approaches that operate on offline datasets, the system performs detection at the point of media consumption by capturing screen and audio data directly from the device.

The system integrates multiple components:

- A frontend Android pipeline for real-time screen and audio capture
- A visual deepfake detection model based on advanced hybrid architectures
- A highly effective audio deepfake detection model based on Wav2Vec2
- A multimodal fusion strategy combining both modalities
- An adaptive learning mechanism that incorporates user feedback

Through iterative experimentation, it was observed that standard CNN and Vision Transformer models were insufficient for real-world deployment due to poor generalization and sensitivity to noise. More advanced architectures that incorporate generative modeling and multi-scale feature extraction demonstrated significantly improved performance.

The audio model emerged as a particularly strong component, providing stable and accurate predictions across a wide range of inputs. When combined with visual detection, the system achieved improved robustness and reliability.

Overall, the system demonstrates that practical deepfake detection requires not only strong models, but also careful system design that considers real-world constraints such as latency, variability in input data, and user interaction.

13.2 Future Work

While the current system performs well, several directions can further improve its capabilities:

- **Database-backed caching:** Introducing a caching layer to store intermediate results and predictions, avoiding redundant computations and improving system efficiency
- **UI/UX improvements:** Enhancing the user interface for better visualization of results, smoother interaction, and more intuitive feedback mechanisms
- **Explainability and transparency:** Providing interpretable outputs (e.g., highlighting manipulated regions or confidence scores) to help users understand model decisions

Future work will focus on making the system more efficient, scalable, and robust to evolving deepfake generation techniques.